DOCUMENTACIÓN TÉCNICA / 03

Guía para añadir contenido

La última visita a casa

Interactuables · Pistas · Puzle · Eventos · Ampliación

Objetivo y referencias del proyecto

La guía muestra cómo reutilizar los componentes ya presentes en la escena Casa para incorporar nuevos objetos, pistas y acontecimientos. Diferencia los cambios realizados desde el Inspector de las ampliaciones que sí exigen nuevas clases C#.

- **Escenas.** Assets/Scenes/Intro.unity es la entrada de la build; Assets/Scenes/Casa.unity contiene el jugador, el escenario y los sistemas de juego.
- **Prefabs de referencia.** Assets/_Prefabs/Character.prefab (FPSController, PlayerInteractor y PlayerInventory), UI.prefab, Key.prefab, Caja/WoodenCrate.prefab y Enviroment/Door.prefab.
- **Contratos reutilizables.** IInteractable define la interacción; AInteractable aporta una base común. IPuzzle, PuzzleManager y GameManager gestionan la resolución y la progresión. Los UnityEvents conectan efectos concretos desde el Inspector.

01 / Crear y configurar un objeto interactuable

Ejemplo real: un recogible basado en la llave

El prefab Key utiliza ItemInteractable y un ItemData para que PlayerInteractor detecte el objeto y PlayerInventory gestione su recogida. La misma estructura permite añadir otros recogibles sin duplicar la lógica del jugador.

1. **Crear la instancia.** Duplicar Assets/_Prefabs/Key.prefab para conservar el original y colocar la nueva instancia en Casa. Cambiar el modelo o sprite si el nuevo objeto necesita otra apariencia.
2. **Configurar detección.** Mantener un Collider activo sobre el objeto y una capa incluida en Raycast Layers de PlayerInteractor. El rayo sale de la cámara del jugador; un collider ausente o excluido impedirá la interacción.
3. **Asignar datos.** Crear o duplicar un ItemData en Assets/_Data/Items/ y enlazarlo en el campo de datos de ItemInteractable. Configurar su identidad y, si corresponde, los parámetros de posición y rotación de agarre.
4. **Configurar el mensaje.** Editar Interaction Prompt en el componente derivado de AInteractable para mostrar la acción contextual al apuntar al objeto.
5. **Guardar y comprobar.** Aplicar los cambios al nuevo prefab y probarlo en Casa. Al interactuar debe recogerse el objeto y PlayerInventory debe registrar la referencia de ItemData.

Qué requiere código

Para una respuesta distinta de leer, recoger o abrir, crear un componente C# que herede de AInteractable e implemente Interact(PlayerInteractor). La clase conserva la integración con IInteractable; no es necesario modificar PlayerInteractor para cada objeto nuevo. El inventario actual solo permite portar un objeto simultáneamente.

02 / Añadir o modificar una pista

Ejemplo real: las notas del salón y la cocina

Las notas separan sus datos del comportamiento. NoteData almacena el contenido y los ajustes visuales; NoteInteractable gestiona la interacción y UIManager abre el mismo panel NoteUI para todas las notas.

1. **Preparar NoteData.** En Assets/_Data/Notes/, duplicar una pista existente o crear un nuevo recurso del mismo tipo. Editar su contenido y, cuando proceda, imagen de fondo, tamaño, posición del texto y parámetros de tipografía.

2. **Colocar la nota.** Duplicar el prefab de nota ya utilizado en Casa, situarlo sobre una superficie y conservar su Collider y NoteInteractable.
3. **Enlazar los datos.** Asignar el nuevo NoteData al campo Note Data del NoteInteractable. El panel común de Assets/_Prefabs/UI.prefab muestra ese contenido sin requerir otra interfaz.
4. **Decidir la reacción al cierre.** Dejar On Closed vacío si se trata solo de información. En la nota de la cocina, este UnityEvent está conectado a TensionEvent.Trigger para iniciar el acontecimiento una única vez.
5. **Comprobar.** Abrir, cerrar y releer la nota; verificar el texto y que el jugador recupera el movimiento al cerrar la interfaz.

Qué requiere código

La creación o edición de notas con los campos existentes no necesita código. Solo habría que modificar NoteData y NoteUI si se incorporase un tipo de contenido o presentación no soportado, como varias páginas o elecciones interactivas.

03 / Configurar la resolución del puzle de la caja

Referencias utilizadas

1. **Identidad y solución.** En el componente BoxPuzzle, establecer un Puzzle Id único y ordenar la lista Solution con los valores en el inspector de BoxButton. Al ser un Enum su configuración es muy sencilla. La versión entregada utiliza Taza → Radio → Lámpara.
2. **Interacción.** Comprobar que BoxInteractable referencia al BoxPuzzle correspondiente y que el objeto mantiene un Collider utilizable por PlayerInteractor.
3. **Interfaz.** En BoxUI del prefab UI, revisar las referencias de los tres botones, los símbolos asociados, las tres imágenes de selección (Slot Images) y Clear Button para borrar la secuencia.
4. **Reacciones visuales y sonoras.** Configurar en BoxPuzzle los UnityEvents On Button Pressed, On Correct y On Failed para los AudioSource correspondientes. On Solved enlaza el cierre de la interfaz, la apertura de la caja y la desactivación de la interacción de sus botones.
5. **Verificación funcional.** Introducir una combinación errónea, comprobar «No era así su rutina» y repetir. Con la combinación correcta, verificar que la caja solo se abre una vez y que la llave debe recogerse manualmente.

Qué requiere código

Reordenar los tres símbolos existentes es configurable desde el Inspector. Para añadir otros símbolos o cambiar la mecánica de entrada hay que ampliar BoxButton, la presentación correspondiente en BoxUI y, si cambia el comportamiento, la implementación de BoxPuzzle. El evento lógico OnSolved se mantiene como contrato con PuzzleManager.

04 / Activar sonido, iluminación o progresión

El proyecto permite configurar eventos de sonido, iluminación y progresión mediante UnityEvents, facilitando la incorporación de nuevas acciones sin modificar los sistemas centrales.

1. Preparar el objeto que realizará la acción.

Crear o seleccionar el GameObject encargado de ejecutar el efecto y añadir el componente correspondiente:

- **Sonido:** AudioSource para reproducir un efecto sonoro.
- **Iluminación:** LightController para controlar el encendido y apagado de una luz.
- **Movimiento:** TransformPositioner para desplazar un objeto entre dos posiciones.
- **Animación:** RotationAnimator para realizar una rotación animada.

2. Configurar los componentes desde el Inspector.

Asignar las referencias necesarias según el comportamiento deseado:

- AudioSource: Audio Clip, Spatial Blend y parámetros de reproducción.
- LightController: Target Light, Audio Source y sonidos de encendido y apagado.
- TransformPositioner: Target y Destination.
- RotationAnimator: parámetros de rotación y duración.

3. Seleccionar el disparador.
Identificar el componente que debe iniciar el evento. Puede tratarse de un objeto interactuable, un puzle o un sistema de progresión que exponga un UnityEvent en el Inspector.

4. Vincular la acción mediante UnityEvents.
En el evento correspondiente, pulsar **+**, arrastrar el GameObject que contiene el componente encargado de ejecutar la acción y seleccionar su método público.

Por ejemplo, pueden utilizarse AudioSource.Play(), LightController.Toggle() o TransformPositioner.ApplyPosition().

Es posible añadir varias acciones a un mismo UnityEvent para ejecutar diferentes efectos a partir de un único disparador.

5. Comprobar el funcionamiento.
Ejecutar la escena y activar el disparador. Verificar que todas las referencias están correctamente asignadas, que los efectos se producen en el momento esperado y que no se repiten accidentalmente.

Eventos nuevos y comunicación entre sistemas

Para añadir efectos visuales o sonoros normalmente basta con configurar los componentes y sus UnityEvents desde el Inspector, sin escribir código adicional.

Si se necesita un comportamiento que los componentes existentes no ofrecen, será necesario crear un nuevo script con un método público que pueda invocarse desde un UnityEvent.

Los cambios relacionados con las **reglas y la progresión del juego** deben comunicarse mediante los eventos C# y los sistemas de coordinación correspondientes, como PuzzleManager y GameManager.

De esta forma, se mantiene separada la lógica de progresión de los efectos audiovisuales y se facilita la ampliación del proyecto.

05 / Ampliación prevista: segundo puzle de rutina

Diseño del nuevo puzle

Como ampliación, antes de acceder a la caja se propone reproducir físicamente la rutina nocturna de la madre. El jugador debe interactuar con tres objetos del escenario en este orden: taza en la cocina, radio en el salón y lámpara del pasillo. Al hacerlo correctamente se muestra la caja que contiene la llave. El puzle de símbolos de la caja permanece como una segunda resolución independiente.

Nuevos componentes C#

- **RoutineObjectInteractable.** Clase derivada de AInteractable (y, por ello, compatible con IInteractable). Almacena una referencia a RoutinePuzzle y un identificador de paso —Taza, Radio o Lámpara—. Su Interact(PlayerInteractor) comunica el paso seleccionado al puzle.
- **RoutinePuzzle.** MonoBehaviour que implementa IPuzzle, con PuzzleId, IsSolved y el evento C# OnSolved. Conserva el índice de progreso de la rutina, comprueba el orden y evita resolver dos veces. Una interacción incorrecta reinicia la secuencia para permitir otro intento.
- **Registro y eventos.** RoutinePuzzle se registra con PuzzleManager.Instance.RegisterPuzzle(this) al iniciarse y se desregistra al destruirse, igual que BoxPuzzle. Al completar el tercer paso, establece IsSolved, emite OnSolved para PuzzleManager y ejecuta un UnityEvent serializado onSolved para mostrar la caja.

Montaje en la escena e Inspector

1. **Crear el controlador.** Añadir un GameObject activo llamado RoutinePuzzle a Casa, asignarle el nuevo componente y establecer un Puzzle Id exclusivo, diferente del utilizado por BoxPuzzle.

2. **Configurar los tres objetos.** Añadir RoutineObjectInteractable a taza, radio y lámpara, junto a sus Colliders. En cada Inspector asignar la misma referencia al controlador RoutinePuzzle y su valor Taza, Radio o Lámpara.
3. **Preparar la caja.** Mantener el BoxPuzzle existente activo para que pueda registrarse con PuzzleManager. Ocultar al comienzo solo la representación y la interacción física de la caja, no el objeto que contiene su lógica de registro.
4. **Enlazar la aparición.** En RoutinePuzzle → On Solved (UnityEvent), conectar la activación de la representación de la caja y de su Collider o mecanismo de interacción. El evento se ejecuta únicamente después de las tres interacciones correctas.
5. **Conservar la condición final.** No modificar GameManager.CanExit(): PuzzleManager ya puede consultar ambos IPuzzle registrados, y DoorInteractable seguirá exigiendo además PlayerInventory.HasItem(requiredKey).

Flujo de comunicación

Taza / Radio / Lámpara → RoutineObjectInteractable → RoutinePuzzle → OnSolved (C#) → PuzzleManager → GameManager. En paralelo, RoutinePuzzle.onSolved (UnityEvent) activa la presentación de la caja; BoxPuzzle se resuelve después con la secuencia de símbolos habitual.

06 / Errores frecuentes y comprobación rápida

- **El objeto no responde.** Revisar Collider, capa incluida en Raycast Layers, distancia de PlayerInteractor, componente interactuable y PlayerInput con la acción Interact.
- **La nota muestra datos antiguos.** Verificar el NoteData asignado a esa instancia, no solo el asset editado. Confirmar que NoteUI y UIManager están presentes en Casa.
- **Un UnityEvent no ocurre.** Comprobar que la instancia correcta está enlazada, el método elegido es público y el GameObject destinatario está activo cuando debe ejecutarse.
- **Un puzle no cuenta para la salida.** Revisar Puzzle Id repetidos, registro en PuzzleManager, IsSolved y emisión del OnSolved de C#. Un UnityEvent visual no sustituye la notificación lógica.
- **La caja oculta bloquea el progreso (Nuevo puzzle añadido).** Evitar desactivar el objeto que aloja BoxPuzzle antes del registro. Mantener activa su lógica y alternar únicamente los objetos visibles o el Collider interactuable.
- **Un evento se repite.** Usar un indicador de activación o resolución y comprobarlo antes de volver a invocar sonidos, animaciones o eventos. TensionEvent y los puzles actuales siguen este criterio.